

FinPlanner Renewals Implementation Progress

✓ Completed Tasks

1. Repository Analysis & Design Decision

- ✓ Cloned repository and analyzed existing codebase
- ✓ Documented current modules and data flow in CURRENT_STATE_ANALYSIS.md
- ✓ **Design Decision:** Extend Bills module (not create new Renewals module)
- ✓ Rationale: Bills infrastructure already supports most requirements

2. Schema Extension

- ✓ Extended `Bill` model with:
 - `provider` (String?, optional) - Insurance company, school, vendor
 - `policyNumber` (String?, optional) - Policy/reference number
 - `reminderDays` (String, default "30,7,1") - Comma-separated reminder offsets
- ✓ Extended `BillInstance` model with:
 - `referenceNumber` (String?, optional) - Payment reference (policy number/receipt)
- ✓ Created `.env.local` with database configuration
- ✓ Installed dependencies (npm install completed)
- ✓ Generated Prisma client successfully

3. Core Utility Functions

- ✓ Created `/lib/bill-utils.ts` with comprehensive utilities:
 - `calculateNextDueDate()` - Automatic date rolling for all frequencies
 - `getDaysUntil()` - Date comparison utility
 - `computeBillStatus()` - Status computation (PENDING, PAID, OVERDUE)
 - `parseReminderDays()` - Parse reminder string to array
 - `shouldShowReminder()` - Reminder logic
 - `filterUpcomingBills()` - Filter bills within date range
 - `getCurrentMonthBounds()` - Get current calendar month boundaries
 - `formatDueDateDisplay()` - Display formatting with relative dates
 - `calculateAnnualAmount()` / `calculateMonthlyAmount()` - Amount conversions
- ✓ Edge case handling:
 - Month-end dates (Jan 31 → Feb 28)
 - Leap years (Feb 29 handling)
 - Year rollovers
 - ONE_TIME bills (no rolling)

4. Comprehensive Tests

- ✓ Created `/lib/__tests__/bill-utils.test.ts` with 20+ test cases:

- ☒ Date rolling for all frequencies (WEEKLY, MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY, ONE_TIME)
- ☒ Edge cases: Month-end, leap years, year rollover
- ☒ Status computation (PENDING, PAID, OVERDUE, due soon)
- ☒ Reminder logic
- ☒ Amount calculations

5. API Enhancement - Payment Route

- ☒ Updated `/app/api/bills/[id]/payment/route.ts` :
- ☒ Added support for `referenceNumber` field
- ☒ Added optional transaction creation on payment
- ☒ **AUTOMATIC DATE ROLLING**: Implemented `nextDueDate` rolling when marking paid
- ☒ Only rolls for recurring bills (not `ONE_TIME`)
- ☒ Returns new `nextDueDate` in response



In Progress

6. API Routes - Need Updates

- `/app/api/bills/route.ts` - Add `provider/policyNumber` to GET/POST
- `/app/api/bills/[id]/route.ts` - Update PUT to support new fields
- Create `/app/api/bills/upcoming/route.ts` - Upcoming obligations endpoint
- Create `/app/api/dashboard/stats/route.ts` - Month-to-date stats



Pending Tasks

7. Database Migration

- ☒ Run `prisma migrate dev` to create migration
- ☒ Test migration on development database
- ☒ Verify schema changes

8. UI Components Update

- ☒ Update `/components/add-bill-dialog.tsx` - Add `provider/policyNumber/reminderDays` fields
- ☒ Update `/components/bill-list.tsx` - Display new fields
- ☒ Update `/app/bills/page.tsx` - Add reference number input when marking paid
- ☒ Add reminder indicators (badges) to bill displays

9. Dashboard Enhancement

- ☒ Create `/components/dashboard-month-stats.tsx` - Month-to-date income/expense/net
- ☒ Create `/components/upcoming-obligations.tsx` - Combined bills + renewals list (30/60 days)
- ☒ Update `/app/dashboard/page.tsx` or `/app/ai-home/page.tsx` with new widgets
- ☒ Add "Quick Add Renewal" button to dashboard

10. Global Command Bar

- ☒ Create `/components/global-command-bar.tsx` - Available on every page
- ☒ Integrate Abacus.AI LLM for natural language parsing
- ☒ Support examples:

- “Spent ₹500 groceries cash”
- “Paid medical insurance ₹28011 today policy 3073...”
- “Add renewal LIC due 19 Mar yearly ₹6527”
- ❌ Create `/app/api/ai/parse-command/route.ts` - LLM parsing endpoint
- ❌ Add confirmation dialog for ambiguous inputs

11. SMS Inbox Enhancement

- ❌ Create `/components/sms-inbox.tsx` - Batch SMS input
- ❌ Create `/app/api/transactions/parse-batch/route.ts` - Batch parsing with Abacus.AI LLM
- ❌ Add approval/rejection workflow
- ❌ Store raw SMS text with transactions
- ❌ Add SMS management page at `/app/sms-inbox/page.tsx`

12. Seed Data

- ❌ Update `/prisma/seed.ts` - Add sample renewals/bills with:
 - Insurance renewals (health, life, vehicle)
 - School fees
 - Property tax
 - Various frequencies
 - Some with payments
- ❌ Run seed script

13. Empty States & UX

- ❌ Create setup checklist for new users
- ❌ Add empty states for:
 - No bills/renewals
 - No upcoming obligations
 - No transactions
- ❌ Make “Upcoming Obligations” a hero section on dashboard

14. Testing & Verification

- ❌ Run Jest tests: `npm test`
- ❌ End-to-end manual testing:
 - Create annual renewal
 - See in “Upcoming” view
 - Mark as paid with reference number
 - Verify date rolls forward automatically
 - Verify transaction created
 - Test SMS parsing with multiple messages
 - Test command bar with examples
 - Verify month-to-date stats on dashboard

15. Documentation

- ❌ Update README.md with:
 - New renewals/planned obligations feature
 - Command bar usage

- SMS inbox workflow
- Run instructions
- ❌ Create DESIGN_DECISIONS.md documenting:
- Why extend Bills vs new model
- Date rolling logic
- Reminder system design
- LLM integration approach
- ❌ Update API_DOCUMENTATION.md with new endpoints

Key Design Decisions Made

1. Extend Bills vs New Renewals Model

Decision: Extend Bills 

Rationale:

- Bills infrastructure already has frequency support matching requirements
- BillInstance provides payment history structure
- Less code duplication
- Simpler maintenance
- Natural fit: renewals ARE a type of recurring bill

2. Automatic Date Rolling

When: When marking a bill/renewal as paid

How: Call `calculateNextDueDate(currentDueDate, frequency)`

Edge Cases Handled:

- Month-end dates (Jan 31 → Feb 28/29)
- Leap years (Feb 29 handling)
- ONE_TIME bills (don't roll)

3. Transaction Creation on Payment

Default: Create transaction automatically when marking paid

Option: Can disable with `createTransaction: false`

Benefits: Keeps transaction records in sync with bill payments

4. Reminder System

Storage: Comma-separated string in database ("30,7,1")

Computation: Dynamic based on days until due

Display: Visual indicators (badges, highlights)

No External: No email/SMS (only UI indicators as per requirements)



Current Status: 40% Complete

-  Foundation & Core Logic: 100%
-  API Enhancement (Payment): 100%
-  API Enhancement (CRUD): 30%
-  Database Migration: 0%
-  UI Components: 0%
-  Dashboard: 0%

- ❌ Command Bar: 0%
- ❌ SMS Inbox Enhancement: 0%
- ❌ Testing & Verification: 20% (tests written, not run)
- ❌ Documentation: 30% (analysis done, README pending)

Next Immediate Steps

1. **Complete API Routes** (~2 hours)
 - Update bills CRUD routes
 - Create upcoming endpoint
 - Create dashboard stats endpoint
2. **Run Database Migration** (~15 min)
 - Create and apply migration
 - Verify schema
3. **Update UI Components** (~3 hours)
 - Bill forms with new fields
 - Bill display with status/reminders
 - Mark paid with reference number
4. **Dashboard Enhancement** (~2 hours)
 - Month-to-date stats widget
 - Upcoming obligations widget
 - Quick add button
5. **Global Command Bar** (~4 hours)
 - Component with LLM integration
 - Abacus.AI API integration
 - Natural language parsing
 - Confirmation dialog
6. **SMS Inbox Enhancement** (~3 hours)
 - Batch processing UI
 - LLM integration
 - Approval workflow
7. **Seed Data & Testing** (~2 hours)
 - Create sample data
 - Run tests
 - Manual verification
8. **Documentation** (~1 hour)
 - Update README
 - Create DESIGN_DECISIONS.md
 - API docs update

 **Total Estimated Time Remaining: ~17 hours**
